

Random maze generation by recursive division, and shortest-path search to the exit

Mohamed Ali Tewfik Hamlil

M1 Artificial Intelligence, Université Grenoble Alpes
Algorithmic Problem Solving — APP1

September 12, 2026

Abstract

A rectangular area of $W \times H$ cells is laid out as a maze by recursive division: a full wall across the longest dimension, exactly one door in it, and the same rule inside each of the two halves. This report states that as a graph problem, shows that the requirements force the maze to be a spanning tree of the grid graph, and derives a generation algorithm together with the data structure it runs on and a proof that what it returns is that spanning tree. Giving every door unit weight makes the length of a route the number of doors crossed, which is what the shortest-path search of Section 4 minimises and what the Manhattan and Euclidean distances are compared against as heuristic guidance. A Python implementation checks the claims on generated instances, and the closing section gives the size of the structure and the cost of both algorithms.

Contents

1	Problem formulation	3
1.1	Statement	3
1.2	From a drawing of walls to a graph	3
1.3	What the specification forces E to be	4
1.4	Constraints on the exit	4
1.5	The two problems, restated on the graph	5
1.6	Conventions	5
1.7	Scope of the deliverable	5
2	Generating the maze into a graph	6
2.1	What has to be stored	6
2.2	The node record	6
2.3	Regions, named by two corners	7
2.4	The recursion stops at a corridor	7
2.5	One division contributes exactly one door	8
2.6	The region tree	8
2.7	One division step	8
2.8	The algorithm	10
2.9	Why the result is a spanning tree	10
2.10	The graph that comes out	11
3	Encoding the distance between adjacent cells as 1	11
4	Shortest path to the exit, with distance as heuristic guidance	11

5	Testing with a Python implementation	11
6	Complexity and analysis of the results	11

1 Problem formulation

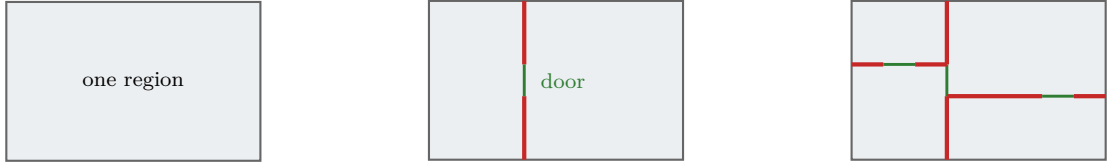
1.1 Statement

A rectangular area of $W \times H$ unit cells is to be laid out as a maze. The layout must be drawn at random, so that W and H do not determine it; the area is enclosed and has a single exit; and it is entered at an arbitrary cell. Two problems follow, and both are to be settled algorithmically before any implementation and then analysed for cost:

1. given the width and the height of the initial area, generate a maze;
2. find the path from any position in the maze to the exit, which the specification states is unique and hence shortest.

The construction is prescribed. Divide the allocated space with a full wall across its longest dimension, leave exactly one door in that wall, and apply the same rule to each of the two spaces this creates. Figure 1 is the entire rule; everything the report derives follows from it.

“At random” constrains one half of the output and not the other. The cells are fixed by W and H : the same WH positions appear in every run, and their coordinates are public knowledge the moment the dimensions are. What must not be predictable is *which doors were kept* — the edge set. The requirement is therefore on the draws, not on the storage: the wall and door positions must come from a genuine random source, because any rule that derives them from the geometry alone (the midpoint, say) makes the maze a function of W and H and reconstructible by anyone holding those two numbers.



(a) A rectangular region, every cell reachable from every other. (b) One full wall across the longest side, pierced by exactly one door. (c) The same rule applied inside each of the two halves.

Figure 1: The construction rule. Each application turns one region into two regions of the same kind joined by a single door, which is what makes this a divide-and-conquer problem and the decomposition a binary tree.

1.2 From a drawing of walls to a graph

A maze drawn on paper is a picture of walls; what an algorithm can generate, store and search is a graph. The translation is the first decision of this report, and every later section rests on it.

Definition 1 (Area, cells, grid graph). Let $W, H \in \mathbb{N}^*$ be the width and the height of the allocated area, measured in unit cells. The cells are

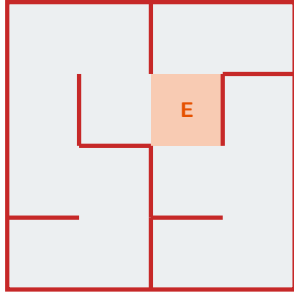
$$V = \{(x, y) \in \mathbb{N}^2 : 0 \leq x < W, 0 \leq y < H\}, \quad |V| = WH,$$

and two cells are *adjacent* when they share a side, that is when $|x - x'| + |y - y'| = 1$. The *grid graph* $G_{\text{grid}} = (V, E_{\text{grid}})$ has the cells as vertices and the adjacent pairs as edges, so

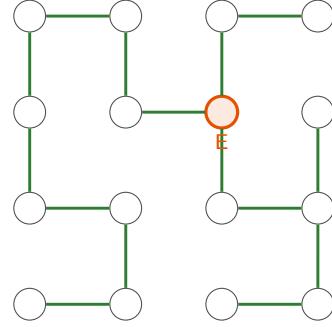
$$|E_{\text{grid}}| = \underbrace{H(W - 1)}_{\text{horizontal}} + \underbrace{W(H - 1)}_{\text{vertical}} = 2WH - W - H.$$

Definition 2 (Maze). A *maze* on that area is a spanning subgraph $M = (V, E)$ with $E \subseteq E_{\text{grid}}$. An edge of E is a *door*: the two cells it joins can be walked between. A pair of $E_{\text{grid}} \setminus E$ is a *wall segment*. One distinguished cell v_{exit} is the exit.

Figure 2 puts the two readings of the same object side by side. The vertex set is fixed by W and H alone; generating a maze therefore means nothing more than *choosing which subset of E_{grid} to keep*.



(a) The maze as walls. The area is enclosed; the exit is a cell, not a gap in the outer wall.



(b) The same maze as a spanning tree of the grid graph.

Figure 2: A maze is not a picture but a graph. Cells become vertices, doors become edges, and a wall is simply an edge of the grid graph that is absent. The exit is a vertex marked on the finished tree — here an interior one, with degree 3 — and neither its position nor its degree affects anything the construction does.

1.3 What the specification forces E to be

Three requirements follow from the statement.

- (R1) **Every cell reaches the exit.** The entry position is arbitrary, so any cell may be a starting point and each must have a route out: M is connected.
- (R2) **There is exactly one exit.** Exactly one cell is designated v_{exit} . It is a *cell*, not a gap in the outer wall: the area is enclosed on every side, and the exit is the one place that leads out of it — a stair or a shaft, in an area dug under a building. It may therefore lie anywhere, including strictly inside.
- (R3) **The route out does not fork.** The path to the exit is required to be unique, hence shortest; a cycle in M would give two distinct simple routes between the cells it passes through, so M is acyclic.

Connected and acyclic on a fixed vertex set is a spanning tree, which settles the shape of the answer before any algorithm is written.

Proposition 1 (The object to produce). *A subgraph $M = (V, E)$ of G_{grid} satisfies (R1) and (R3) if and only if it is a spanning tree of G_{grid} . It then has $|E| = WH - 1$ doors, and between any two cells there is exactly one simple path, which is therefore also a shortest one.*

That last clause is what makes the second problem well posed: asking for *the* path out is only meaningful because there is one. It is also the lever for the rest of the report — Section 2 shows that the construction rule produces such a tree *by construction* rather than by checking afterwards.

1.4 Constraints on the exit

Three questions about v_{exit} are worth settling before an algorithm is written. None of the three turns out to constrain the construction, but two of them invite an answer that would.

When is it chosen — before or after generation?

After, and it can only be after in any useful sense. Generation takes W and H and nothing else; by Proposition 1 the maze it returns makes every cell reachable from every other, so *any* cell designated afterwards is reachable from all of them. Choosing first would change nothing about what is built, because there is no step in the construction that could consult it. Section 2 never does.

Must it be on the boundary?

No. This is the tempting answer and it is wrong. The exit is a cell of the area, not a hole in the wall enclosing it, so nothing obliges it to touch the perimeter — and the subject’s own worked example puts it several cells inside one, with the outer wall drawn closed the whole way round. An area dug beneath a building leaves it by a stair, and a stair can be anywhere.

Must it have exactly one door?

No, and requiring it would be a mistake. Uniqueness of the route out comes from M being a tree, not from the exit’s degree: whether v_{exit} has 1, 2, 3 or 4 doors, there is still exactly one simple path to it from anywhere. Worse, the constraint can be unsatisfiable — a spanning tree of G_{grid} need not place any of its leaves where one is wanted, so demanding a degree-one exit at a chosen position can rule out a maze the construction legitimately produced.

So the exit carries no constraint at all: it is a label applied to a finished tree, and every cell is an equally valid candidate. The report treats it as one, and Figure 2 marks an interior cell of degree 3 to make the point.

1.5 The two problems, restated on the graph

Problem 1 — Generation.

Given W , H and a source of random bits, produce a spanning tree M of G_{grid} , drawn at random so that no two runs need agree, together with the structure that records how it was built. The exit is not an input: any cell is designated v_{exit} once M exists.

Problem 2 — Escape.

Given M and any start cell $s \in V$, return the unique simple path from s to v_{exit} .

Problem 1 is solved in Section 2. Problem 2 needs a notion of length before it can be measured: Section 3 gives every door weight 1, and Section 4 runs the shortest-path search that weight makes meaningful, guided by the Manhattan and Euclidean distances.

1.6 Conventions

Symbol	Meaning
W, H	width and height of the area, in cells
$n = WH$	number of cells, the size of the input
(x, y)	cell in column x (left to right) and row y (top to bottom)
$G_{\text{grid}}, E_{\text{grid}}$	the grid graph and its edges: every adjacency, all doors open
$M = (V, E)$	the maze produced: a spanning tree of G_{grid}
v_{exit}	the single exit cell, anywhere in V
$(x_0, y_0) - (x_1, y_1)$	a region, by its upper-left and lower-right cells

1.7 Scope of the deliverable

The specification fixes the contents and not only the topic: a data structure with a statement of what it holds, an algorithm for each of the two problems, the size of that structure and the cost of both algorithms. The table below is the report’s own checklist against that list.

Required	Where	State
Data structure, and what it stores	Section 2	done
Maze generation algorithm	Section 2	done
Cost of a door, and what a path costs	Section 3	to write
Path-finding algorithm	Section 4	to write
Implementation (optional in the specification)	Section 5	to write
Size of the structure, complexity of both algorithms	Section 6	to write

2 Generating the maze into a graph

Section 1 fixed the target: a spanning tree of G_{grid} , drawn at random. This section builds one. The structure comes first, because the algorithm is written against it, and the guarantee comes last, because it is the construction rule that supplies it.

2.1 What has to be stored

Two different things are worth keeping, and confusing them is the usual way to end up storing neither well.

The maze itself — which doors exist.

It is what a walker needs, and the only thing needed to answer Problem 2.

The way it was built — the region decomposition.

Which wall was put where, and in what order. It is the binary tree the divide-and-conquer rule produces for free, and the object whose depth the cost analysis of Section 6 is written in terms of.

2.2 The node record

The maze is a set of nodes, one per cell. A node holds

$$\text{Node} = \langle \underbrace{\text{value}}_{\text{its } (x,y) \text{ coordinates}}, \underbrace{\text{neighbours}}_{\text{the cells it has a door to}} \rangle,$$

and the neighbour list is *empty when the node is created*. A cell has four sides, so the list never holds more than four entries, and it grows only when the construction puts a door on that node. An index $\langle P_i, P_j \rangle$ — a $W \times H$ array of references — maps a coordinate pair to its node in $O(1)$, so a division can reach the cells it wants without searching. Figure 3 is one node before and after.

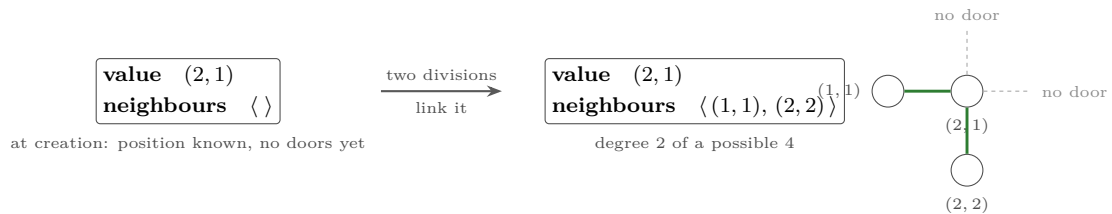


Figure 3: The node record. A node carries its coordinates and a list of the neighbours it has a door to; the list is empty when the node is made and is appended to only by a division whose door lands on that node. A cell has four sides, so the list never exceeds four entries, and in the finished maze the entries across all nodes number $2(WH - 1)$ — each door recorded once at each of its two ends.

Each door is recorded at both of its ends, so the finished maze holds $2(WH - 1)$ list entries for $WH - 1$ doors. That is the cost of being able to enumerate a cell's exits in $O(\deg)$ from the cell alone, which is what the search of Section 4 does at every step.

Remark. An alternative is to give each cell two links, to its right and lower neighbour, so that every *possible* door is owned by exactly one cell and none is stored twice. That trick exists to stop a dense grid of candidate edges being duplicated. It has nothing to save here: the construction below never represents a door that does not exist, so there is no duplication to avoid and no NIL to store for a wall.

Remark. Storing the coordinates costs nothing in secrecy. The cells and their positions are fixed by W and H , and are known to anyone who knows the dimensions; what the randomness has to protect is the edge set, and a node's *value* reveals nothing about which of its four sides became doors. The list does, but the list *is* the maze.

2.3 Regions, named by two corners

A region is a rectangle of cells, and the algorithm names it by the two cells at its ends: $(x_0, y_0) - (x_1, y_1)$, upper-left and lower-right, with $x_0 \leq x_1$ and $y_0 \leq y_1$. The whole area is $(0, 0) - (W - 1, H - 1)$. In the pseudocode those two cells are called p and q , and a cell's two coordinates are reached by index: $p[0]$ is its x and $p[1]$ its y . Indexing rather than naming them is what lets the division below be written once for both axes.

This is the same currency the rest of the program is written in. A node's *value* is an (x, y) pair, a door is a pair of them, and now a region is too, so there is one coordinate type throughout and no second convention to keep straight — which is also how the drawing routine provided with the subject speaks, taking a rectangle as two corner points.

Both corners are *cells of the region*, not fences around it. A region therefore spans $x_1 - x_0 + 1$ columns and $y_1 - y_0 + 1$ rows, and one column wide is $x_0 = x_1$ — a difference of *zero*. A difference of one, $x_1 - x_0 = 1$, is a region two columns wide, which is not a corridor and still has a wall to place. Naming a region by a half-open range instead, with q one cell past the last, would make the width $x_1 - x_0$ and one-wide $x_1 - x_0 = 1$; it is rejected because q would then not be a cell, which puts back exactly the second convention the corner form removed.

2.4 The recursion stops at a corridor

Definition 3 (Corridor). A region is a *corridor* when one of its two sides is a single cell: $x_0 = x_1$, one column wide and any height, or $y_0 = y_1$, one row tall and any width. A single cell, where both hold, is the smallest corridor.

The name is worth the trouble because a corridor is the case where the construction has nothing to decide. Its cells lie in a line, so the subgraph G_{grid} induces on it is already a path; a path on m vertices has $m - 1$ edges and is a tree, so it is its own and only spanning tree. Put a wall across a corridor and it has exactly one crossing, which is then forced to be the door — the division would change nothing. Dividing further is therefore not wrong, merely pointless, and the recursion stops there, handing the region to **SpanCorridor**, which lays those forced doors and returns the leaf that records them.

Stopping at a corridor rather than at a single cell is not only economy, though it is substantial economy: on a 64×64 area it took 1301 calls against 8191, and a 1×40 area is finished in a single call instead of 79. The real payoff is that *every region reaching a division now has both sides at least 2*. Both axes therefore admit a wall, always, and the orientation can be a plain coin with no feasibility test in front of it. The degenerate case is not guarded against; it is removed.

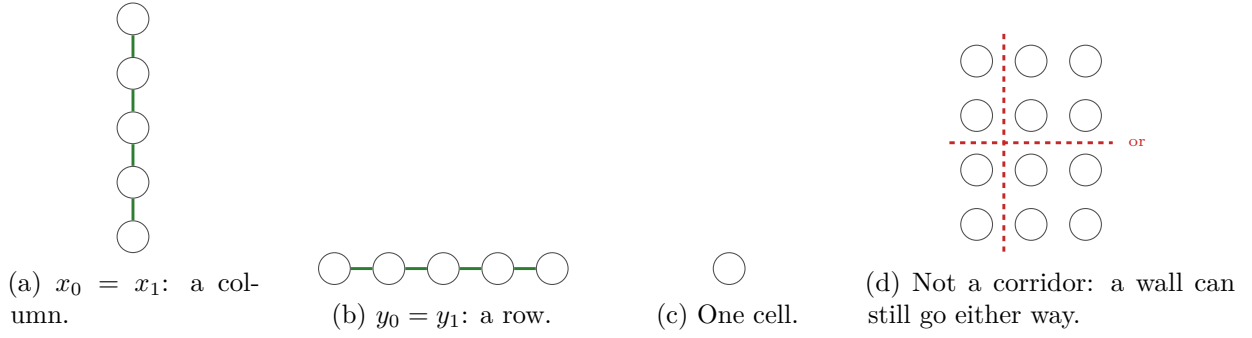


Figure 4: A *corridor* is a region one cell thick, in either direction. Its cells lie in a line, so the induced graph is already a path and there is exactly one way to span it — nothing about a corridor is left to decide, which is why the recursion stops at one. A region thicker than that, on the right, still has both a choice of orientation and a choice of position.

2.5 One division contributes exactly one door

Take any door $e = (u, v)$ of the finished maze and walk down the region tree from the root. At each internal node, u and v either fall in the same child — and the walk continues — or they are separated, which happens precisely when e crosses that node’s wall, and then e is that node’s door, being the only crossing that survived. Otherwise the walk ends at a leaf holding both, and e is one of the doors that leaf laid down along its corridor.

So the doors split cleanly in two: one per internal node, plus the internal doors of each corridor leaf. Nothing is counted twice and nothing is missed, which is what licenses the construction to run *forwards* — a division adds its own door and nothing else, a leaf adds its corridor, and a wall is never represented at all. It is simply the doors that were never added.

Remark. This is what makes the neighbour list a sound representation. A list cannot express “these two cells are not joined” except by omission, which is useless if the algorithm needs to remove links; here nothing is ever removed, so omission is exactly right.

2.6 The region tree

The recursion’s shape is a binary tree, one node per call:

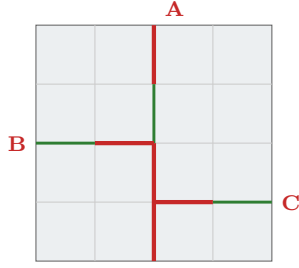
- an **internal node** records one division — its orientation t , the boundary j the wall sits on, the position i of its single door — and has the two regions the division created as its children;
- a **leaf** is a corridor, of one cell or many.

A binary tree with L leaves has $L - 1$ internal nodes, so the $WH - 1$ doors are $L - 1$ division doors and $WH - L$ corridor doors. Figure 5 shows the first three divisions of a 4×4 area beside the top of the tree they build.

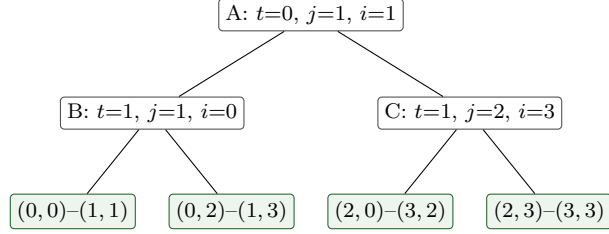
2.7 One division step

A call handles a region $(x_0, y_0) - (x_1, y_1)$. If it is a corridor (Definition 3) Section 2.4 disposes of it. Otherwise both sides are at least 2 and:

1. **Toss for the axis t to cut.** Both admit a wall, so the coin is free. Write $t = 0$ for the x axis and $t = 1$ for the y axis, and write $p[t]$ for the coordinate of cell p on axis t ; then $1 - t$ is the other axis, and one piece of code covers both cases.
2. **Choose the wall position j on axis t ,** uniformly in $[p[t], q[t] - 1]$, and **the door position i on the other axis,** uniformly in $[p[1 - t], q[1 - t]]$.



(a) The first three divisions of a 4×4 area.



(b) The top of the region tree it builds. The four boxes at the bottom are regions still to be cut, not cells.

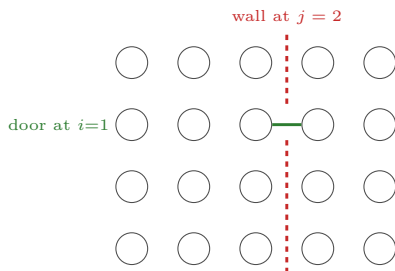
Figure 5: Every division is one internal node. The node records the orientation t , the wall position j and the door position i it drew, and its two children are the two regions it created; a leaf is a corridor. A tree with L leaves has $L - 1$ internal nodes, so of the $WH - 1$ doors of Proposition 1, $L - 1$ are division doors and the rest are laid down inside the corridors.

3. **Link the two cells the door falls on**, appending each to the other's neighbour list.

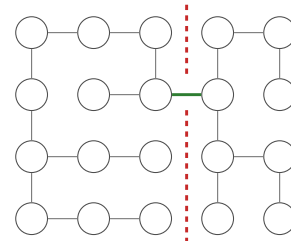
4. **Recurse** on the two regions — the same corners with the axis- t coordinate cut at j — and hang them under the node recording $\langle t, j, i \rangle$.

Remark (Why one branch and not two). Written out longhand, the vertical case and the horizontal case are the same six lines with x and y exchanged. Indexing the axis collapses them into one, which is worth doing for a reason beyond brevity: two mirrored copies are where a transcription slip hides, since each reads correctly on its own and only the comparison between them shows the error. One copy has nothing to disagree with.

Remark (Departure from the specification). The specification puts the wall across the *longest* dimension; this report tosses for it, which is a deliberate deviation and worth naming. The longest-dimension rule keeps regions square, so it is the more disciplined choice; a free coin lets regions grow long and thin and makes the maze more corridor-like. It costs little in depth, because depth is driven by the uniform *position* of the wall rather than by its orientation, and correctness is untouched: Proposition 2 never uses which axis was cut, only that one was.



(a) The call links one pair and stops. Every other node is still isolated; no crossing was examined, let alone removed.



(b) After both recursive calls return. Each half is a tree, and the door is still the only edge between them.

Figure 6: One division, with $t = 0$ — the coin named the x axis, so the wall is vertical. The call draws the wall position j on that axis and the door position i on the other, links the two cells the door falls on, and recurses on the two halves. Counting edges: 11 on the left, 7 on the right, one door, total $19 = 20 - 1$ on 20 cells — the tally of Proposition 2.

2.8 The algorithm

Algorithm 1: Maze generation by recursive division.

Input: $W, H \geq 1$, the width and height of the area in cells. The bound is a precondition, not a convention: at $W = 0$ the region $(0, 0) - (-1, H - 1)$ holds no cell and there is nothing for the first draw to land on.

Output: N , the set of nodes holding a spanning tree of G_{grid} ; r , the root of the region tree that built it.

```

1 Function GenerateMaze( $W, H$ ):
2    $N \leftarrow \{ \text{NewNode}(x, y) : 0 \leq x < W, 0 \leq y < H \}$ ; // every neighbour list empty
3    $r \leftarrow \text{Divide}(N, (0, 0), (W - 1, H - 1))$ ;
4   return ( $N, r$ );

5 Function Divide( $N, p, q$ ):
6   if  $p[0] = q[0]$  or  $p[1] = q[1]$  then return SpanCorridor( $N, p, q$ );
7    $t \leftarrow \text{RandomBit}()$ ; // the axis to cut: 0 is  $x$ , 1 is  $y$ 
8    $j \leftarrow \text{RandomIn}(p[t], q[t] - 1)$ ; // wall between  $j$  and  $j + 1$  on axis  $t$ 
9    $i \leftarrow \text{RandomIn}(p[1-t], q[1-t])$ ; // the one door, at  $i$  on the other axis
10   $\text{Link}(N, \text{CellAt}(t, j, i), \text{CellAt}(t, j+1, i))$ ;
11   $\ell \leftarrow \text{Divide}(N, p, \text{WithAxis}(q, t, j))$ ;
12   $r \leftarrow \text{Divide}(N, \text{WithAxis}(p, t, j+1), q)$ ;
13  return CutNode( $t, j, i, \ell, r$ );

14 Function SpanCorridor( $N, p, q$ ):
15  for  $x \leftarrow p[0]$  to  $q[0] - 1$  do  $\text{Link}(N, (x, p[1]), (x+1, p[1]))$ ;
16  for  $y \leftarrow p[1]$  to  $q[1] - 1$  do  $\text{Link}(N, (p[0], y), (p[0], y+1))$ ;
17  return Leaf( $p, q$ );

18 Function Link( $N, u, v$ ):
19   $\text{append } N[v] \text{ to } N[u].\text{neighbours}$ ;
20   $\text{append } N[u] \text{ to } N[v].\text{neighbours}$ ;

21 Function CellAt( $t, u, v$ ):
22  return the cell whose axis- $t$  coordinate is  $u$  and whose other is  $v$ ;

23 Function WithAxis( $p, t, u$ ):
24  return the cell  $p$  with its axis- $t$  coordinate replaced by  $u$ ;
```

`SpanCorridor()` is the whole of the base case: it lays the corridor's forced doors *and* returns the leaf that records the region, so what used to be two lines that had to be kept in step is one call. It needs no case analysis either — on a corridor one of its loops is empty by construction and on a single cell both are, so the same two loops cover a row, a column and one cell.

`CellAt()` and `WithAxis()` are the only price of indexing the axis, one line each, and they are what let the division be written once rather than twice.

Every `Link()` joins two cells of the region that called it, and they are adjacent — so every appended entry is a genuine edge of G_{grid} .

The exit plays no part in generation, for the reasons given in Section 1.4: the algorithm never reads it, and any cell at all may be designated once the tree exists.

2.9 Why the result is a spanning tree

Proposition 2 (Correctness of Algorithm 1). *On a region of m cells, Divide leaves the subgraph induced on that region connected and acyclic, with exactly $m - 1$ doors. In particular GenerateMaze returns a spanning tree of G_{grid} .*

Proof. By induction on m . If the region has a side of one it is a corridor of m cells in a line, and **SpanCorridor** joins each to the next: a path, which is connected and acyclic and carries $m - 1$ doors. This covers $m = 1$ too, where both loops are empty and $m - 1 = 0$.

Otherwise both sides are at least 2, so $q[0] - p[0] \geq 1$ and $q[1] - p[1] \geq 1$: whichever axis the coin names, $q[t] - p[t] \geq 1$, so j is drawn from a non-empty range and the two sub-regions R_ℓ and R_r are non-empty, with $m_\ell + m_r = m$. The call adds the single door at i , whose two cells lie one in each sub-region, and then recurses; by the induction hypothesis the two calls leave their regions connected and acyclic with $m_\ell - 1$ and $m_r - 1$ doors. Neither recursive call can add an edge between the two sub-regions, because every **Link** a call makes joins two cells of its own region.

So the door at i is the only edge between R_ℓ and R_r . Joining two disjoint trees by a single edge yields a tree: connected, since any two cells are joined inside their own half or by crossing that door; acyclic, since a cycle would have to cross the wall an even, hence non-zero, number of times while only one crossing exists. The door count is $(m_\ell - 1) + (m_r - 1) + 1 = m - 1$.

Applying this to the whole area gives a connected acyclic spanning subgraph of G_{grid} on all WH cells: a spanning tree, with $WH - 1$ doors. $\square$

The guarantee is therefore structural. Nothing is checked after the fact, no attempt is ever rejected and retried, and the uniqueness of the route out — the whole of Problem 2 — is a consequence of the shape of the recursion rather than of a property that had to be tested.

2.10 The graph that comes out

Generation returns the set N of nodes, each carrying its coordinates and the doors it ended up with, together with the root r of the region tree. Three readings of that one result, none of them a separate copy:

View	Held as	Used by
Adjacency	each node's neighbour list	the search of Section 4
Position	each node's <i>value</i>	the heuristics of Section 4
How it was built	the region tree rooted at r	the cost analysis of Section 6

The second row is what makes Section 4 possible at all: a heuristic that estimates the remaining distance to the exit needs the coordinates of the cell it is standing on, and they are in the node.

Figure 2b is the output of one 4×4 run, whose first three divisions are drawn in Figure 5a: 15 doors on 16 cells, one path between any two of them. What Section 3 adds is a length on those doors, so that “the path out” becomes a quantity a search can minimise.

3 Encoding the distance between adjacent cells as 1

To be written.

4 Shortest path to the exit, with distance as heuristic guidance

To be written.

5 Testing with a Python implementation

To be written.

6 Complexity and analysis of the results

To be written.